

開始在 Antigravity 使用 Spec Driven Development

來源：<https://codelabs.developers.google.com/codelabs/getting-started-with-spec-driven-development-in-antigravity?hl=zh-tw>

步驟數：11

在本實作實驗室中，您將瞭解如何使用 Antigravity 建立應用程式，並部署至 Google Cloud。

目錄

1. [本實驗室的目標](#)
2. [環境設定](#)
3. [專案設定](#)
4. [啟用 API](#)
5. [確認是否已套用抵免額](#)
6. [Google Antigravity 簡介](#)
7. [AI 規格導向開發簡介](#)
8. [使用 Google Antigravity 開發網頁應用程式](#)
9. [設定部署環境](#)
10. [清除所用資源](#)
11. [結論](#)

步驟 1 · 約 0 分鐘

1. 本實驗室的目標

在本實作實驗室中，您將瞭解如何使用 Antigravity 建立應用程式 (使用 [Google Antigravity](#))，並將其部署至 Google Cloud。本實驗室也會介紹規格導向開發的概念。

課程內容

- 瞭解 [Google Antigravity](#) 的基本概念。
- 瞭解規格導向開發的基本概念
- 瞭解如何輕鬆在 [Cloud Run](#) 中部署應用程式。



圖 1：Antigravity 是 Google 開發的代理優先開發工具。

步驟 2 · 約 0 分鐘

2. 環境設定

1. 安裝 Antigraity :

👉 Download the [Google Antigraity] (<https://antigravity.google/docs/get-started>) for your environment from [here] (<https://antigravity.google/>).

👉 在環境中安裝 Antigraity。

👉 前往 Antigraity 的安裝資料夾，然後按兩下開啟安裝程式。

👉 按照安裝程式操作說明，在環境中安裝 Antigraity。

2. 安裝 Python

👉 前往 <https://www.python.org/downloads/>，為您的系統安裝 Python。

3. 安裝 gcloud

👉 gcloud 是一種指令列工具，可讓您在 Google Cloud 上執行各種作業。請按照[這裡](#)的操作說明，在環境中安裝 gcloud。

👉 安裝完成後，請開啟系統終端機並輸入 **gcloud**，測試安裝是否成功。

```
Command name argument expected.

Available command groups for gcloud:

AI and Machine Learning
  ai                Manage entities in Vertex AI.
  ai-platform       Manage AI Platform jobs and models.
  colab             Manage Colab Enterprise resources.
  gemini            Manage resources associated with Gemini Code
                   Assist and Gemini Cloud Assist.
  ml                Use Google Cloud machine learning capabilities.
  notebooks         Notebooks Command Group.
  workbench         Workbench Command Group.

API Platform and Ecosystems
  api-gateway       Manage Cloud API Gateway resources.
  apigee            Manage Apigee resources.
  endpoints         Create, enable and manage API services.
  recommender       Manage Cloud recommendations and recommendation
                   rules.
  services          List, enable and disable APIs and services.

Anthos CLI
  anthos            Anthos command Group.

Batch
  batch             Manage Batch resources.

Big Data
```

圖 2：安裝 gcloud 後，您可以在終端機中輸入 gcloud，測試安裝結果

3. 專案設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

- 如果沒有可用的專案，請在 [GCP 主控台](#) 中建立新專案。在專案選取器 (Google Cloud 控制台左上角) 中選取專案

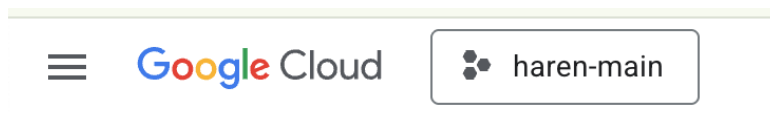


圖 2：按一下 Google Cloud 標誌旁的方塊，即可選取專案。確認已選取專案。

- 在本實驗室中，我們將使用 [Cloud Shell 編輯器](#) 執行工作。開啟 Cloud Shell，並使用 Cloud Shell 設定專案。
- 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
- 如果尚未開啟終端機，請依序點選選單中的「Terminal」>「New Terminal」。您可以在這個終端機中執行本教學課程的所有指令。
- 您可以在 Cloud Shell 終端機中執行下列指令，檢查專案是否已通過驗證。

```
gcloud auth list
```

- 在 Cloud Shell 中執行下列指令，確認專案

```
gcloud config list project
```

- 複製專案 ID，然後使用下列指令設定

```
gcloud config set project <YOUR_PROJECT_ID>
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list
```

步驟 4 · 約 0 分鐘

4. 啟用 API

我們需要啟用一些 API 服務，才能執行本實驗室。在 Cloud Shell 中執行下列指令。

```
gcloud services enable aiplatform.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
```

API 簡介

- [Vertex AI API](#) (`aiplatform.googleapis.com`) 可存取 [Vertex AI](#) 平台，讓應用程式與 Gemini 模型互動，生成文字、進行聊天工作階段及呼叫函式。
 - [Cloud Resource Manager API](#) (`cloudresourcemanager.googleapis.com`) 可讓您以程式輔助方式管理 Google Cloud 專案的中繼資料，例如專案 ID 和名稱。其他工具和 SDK 通常需要這些資料，才能驗證專案身分和權限。
-

5. 確認是否已套用抵免額

在「專案設定」階段，您已申請免費抵免額，可使用 Google Cloud 服務。套用抵免額後，系統會建立名為「Google Cloud Platform Trial Billing Account」的新免費帳單帳戶。如要確認抵免額已套用，請在 [Cloud Shell 編輯器](#) 中按照下列步驟操作：

```
curl -s https://raw.githubusercontent.com/haren-bh/gcpbillingactivate/main/activate.py |  
python3
```

如果成功，您應該會看到如下的結果：如果看到「Successfully linked project」(專案連結成功)，表示帳單帳戶設定正確。執行上述步驟後，系統會檢查你的帳戶是否已連結，如果沒有，系統會為你連結。如果您尚未選取專案，系統會提示您選擇專案，您也可以事先按照專案設定中的步驟操作。

```
Finding an active billing account...  
Found billing account: Google Cloud Platform Trial Billing Account (014890-08C583-49ACEE)  
Finding current project...  
Using project: testproject1-482302  
Linking project testproject1-482302 to billing account Google Cloud Platform Trial Billing Account (014890-08C583-49ACEE)...  
Successfully linked project.
```

圖 3：確認已連結帳單帳戶

6. Google Antigravity 簡介

Google Antigravity 是 Google DeepMind 開發的 AI 優先軟體開發工具，Google Antigravity 運用長期累積的軟體開發專業知識和尖端 AI 技術，為開發人員提供流暢無縫的 AI 輔助開發體驗。

以下列舉 Google Antigravity 的幾項主要功能。

下圖顯示 Google Antigravity 的基本元素。

- 1. 🖱️ 開啟瀏覽器，開始探索瀏覽器的各個部分。

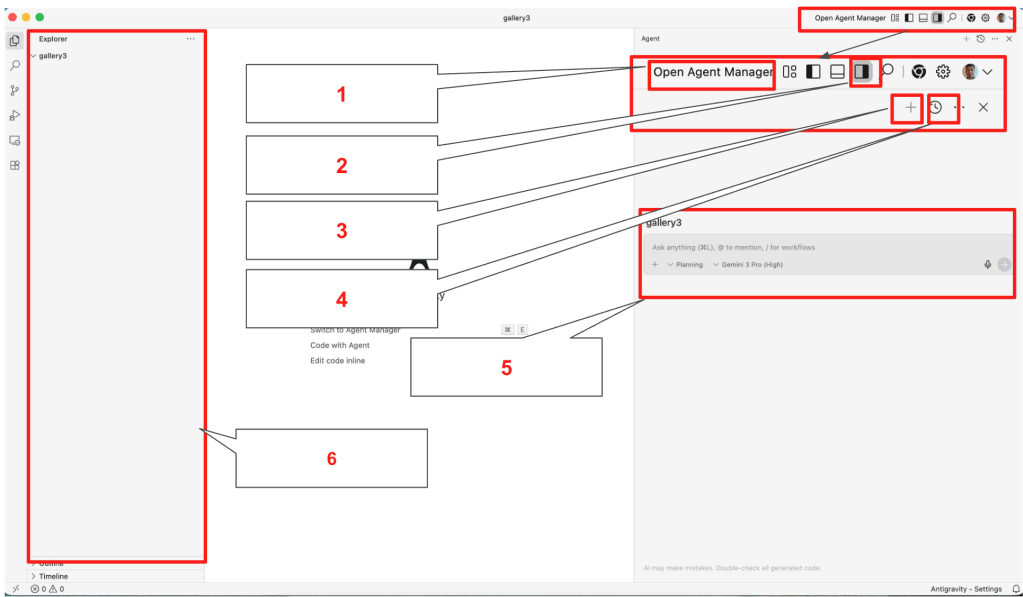


圖 4：Google Antigravity 的基本元素，詳情請參閱表 1

表 1：Google Antigravity 的基本元件詳細資料

Component Index	Component Name	Function
1	Agent Manager	Provide access to your agent manager where you can manage all your agents
2	Toggle Agent Pane	Toggles and untoggles your agent pane
3	New Session	Starts a new Agent Conversation while keeping old ones separately.
4	Past Conversations	Retrieve previous conversations
5	Agent Pane	The agent pane where you can have conversation with the AI agent

6	Explorer	File explorer
---	----------	---------------

1. 內建 **Gemini 3** 和 **Nanobanana** 模型：透過 [Google Antigravity](#)，您可以使用 Google 最新的旗艦模型，例如 Gemini 3 和 Nanobanana。除了這些模型，您也可以使用 Claude 等第三方模型。

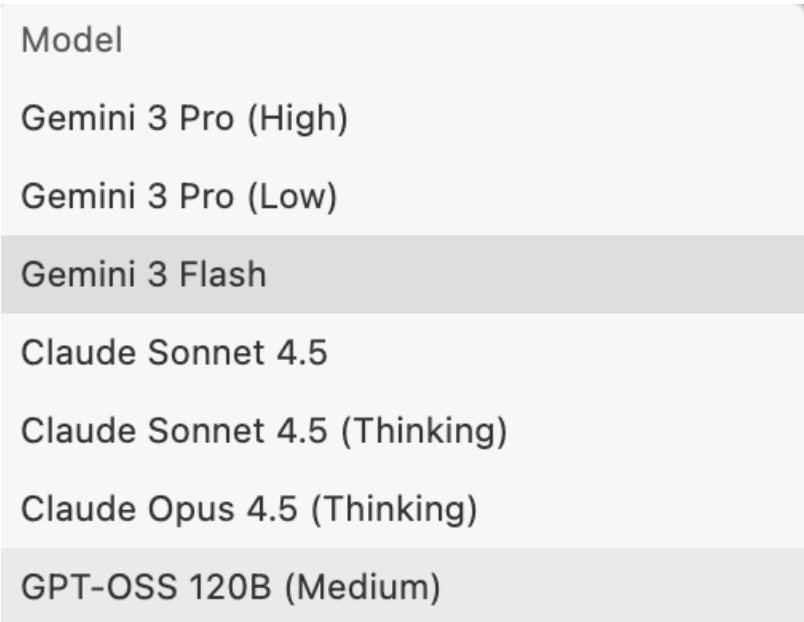


圖 5：您可以在 Google Antigravity 2 中使用多種模型。以代理為導向的程式碼編寫：Antigravity 提供以代理為導向的原生程式碼編寫體驗，讓開發人員保持生產力，不會受到干擾。

- 1. **規劃和全面使用者控制**：代理程式會根據您的輸入內容將工作轉換為計畫，並在執行前要求您核准。確保使用者可以在執行工作前隨時變更代理程式的方向。
- 2. **使用者意見回饋**：在代理程式執行期間，如果使用者需要向代理程式提供額外指令，可以提供意見回饋。
- 3. **多個代理**：您可以生成多個代理，同時處理不同工作。舉例來說，代理程式 A 可以重構驗證邏輯，代理程式 B 則為新的 API 編寫單元測試，代理程式 C 則在背景研究程式庫。
- 4. **編輯器、終端機和瀏覽器中的代理程式**：[Google Antigravity](#) 代理程式可在多個介面運作。
- 5. **編輯器**：[Google Antigravity](#) 代理程式會編寫程式碼，並在編輯器中向您顯示編寫的程式碼。
- 6. **終端機**：視工作而定，[Google Antigravity](#) 代理程式可能需要存取終端機，才能執行部分指令。代理程式會在需要時為您執行指令。
- 7. **瀏覽器**：代理程式也可以與瀏覽器搭配使用。如果您需要測試網路應用程式，這項功能就特別實用，因為代理程式可以在網路瀏覽器中執行應用程式、測試及偵錯。

7. AI 規格導向開發簡介

規格導向開發是新興的軟體工程典範，將結構化規格和 AI 代理置於開發生命週期的核心。與基本 AI 程式設計中常見的「提示和修補」(試誤) 方法不同，SDD 會優先收集詳細需求、設計系統/架構，以及規劃測試。它借用**瀑布式模型**設計階段的嚴謹性，但透過自動化將其整合到現代化的**敏捷疊代迴圈**中。雖然這個程序需要您事先進行細密的規劃和記錄，但實際上是疊代程序，因為 AI 代理程式可快速實作和測試。這樣一來，您就能更快取得意見回饋，並據此改善文件。

核心理念

在這個模式中，人類工程師會從「程式碼撰寫者」轉變為「系統架構師」。人類的主要責任是提供問題和解決方案的詳細說明。這項詳細輸出內容是 **Single Source of Truth (SSOT)**，AI 代理程式會使用這項內容生成、驗證及調整程式碼集。

SDD 生命週期

這項程序包含下列元件。步驟 1 至 3 主要以人為中心，步驟 4 至 5 則以 AI 代理程式為中心。這是一個反覆的過程，在週期結束後，意見回饋可用於改善規格。

1. **收集需求**：明確找出商業邏輯、使用者需求和系統限制。
2. **架構設計**：定義系統結構、資料模型和整合點。
3. **系統與測試規格**：建立機器可讀取 (或高度結構化) 的文件，定義系統的**功能**和**驗證方式**。
4. **自動生成程式碼**：AI 代理程式會根據規格生成可直接用於正式環境的程式碼。
5. **測試與驗證**：自動化套件會根據測試規格驗證產生的程式碼。

主要營運原則

1. 設計與實作迴圈

步驟 1 到 5 並非線性路徑，而是**持續進行的意見回饋循環機制**。由於程式碼生成 (步驟 4) 和測試 (步驟 5) 大多是自動執行，工程團隊可以將大部分頻寬轉移到前三個階段。發現錯誤或功能變更時，工程師會更新「規格」，而不是程式碼，並再次觸發迴圈。

2. 模組化精細程度

為維護系統完整性，SDD 必須套用至**細部模組**，而非單體區塊。

- **隔離**：如果特定模組驗證失敗，只需要重新指定並重新產生該模組。
- **可擴充性**：定義明確的小型模組可避免 AI 「幻覺」，並確保 AI 代理程式的脈絡視窗保持專注和準確。

3. 品質驗證 (QC)

在這個範例中，「系統規格」是藍圖，「測試規格」是評估標準。測試規格可確保生成的程式碼一律符合預先決定的品質要求。整個程序可無縫整合至現有的 CI/CD 管道，確保整體系統健康狀態也符合品質要求。

在本實驗室中，我們將使用 [Google Antigravity](#) 探索規格導向開發的基本知識

8. 使用 Google Antigravity 開發網頁應用程式

在本實驗室中，我們會建立簡單的相片庫應用程式。圖像生成模型 Nanobanana 內建於 [Google Antigravity](#)。我們會使用 Nanobanana 建立必要圖片。

設定網路瀏覽器

網頁瀏覽器將用於自動測試應用程式。在下列步驟中，我們將設定瀏覽器，讓 **Antigravity** 自動測試應用程式。

1. 🖱️ 按一下右上角的「設定」按鈕 (齒輪圖示)，然後選取「Open Antigravity User Settings」(開啟 Antigravity 使用者設定)
2. 🖱️ 按一下左窗格中的「代理程式」，然後在「ARTIFACT」部分中，選取「一律繼續」

ARTIFACT

Review Policy

Specifies Agent's behavior when asking for review on artifacts, which are documents it creates to enable a richer conversation experience.

- Always Proceed - Agent never asks for review. This maximizes the autonomy of the Agent, but also has the highest risk of the Agent operating over unsafe or injected Artifact content.
- Agent Decides - Agent will decide when to ask for review based on task complexity and user preference.
- Request Review - Agent always asks for review.

Always Proceed ▾

，即可查看政策。

3. 🖱️ 按一下左側窗格中的「瀏覽器」，確認已啟用「啟用瀏覽器工具」

GENERAL

Enable Browser Tools

When enabled, Agent can use browser tools to open URLs, read web pages, and interact with browser content. This allows the Agent access to important (and often critical) knowledge and methods of validation, but any browser integration does increase exposure to external malicious parties for security exploits.



。

使用 Google Antigravity 建立應用程式

1. 🖱️ 按一下「Google Antigravity」圖示，開啟 [Google Antigravity](#)
2. 🖱️ 在個人資料夾中建立名為「**Gallery**」的資料夾，例如 電腦。
3. 🖱️ 在 Antigravity 中按下「開啟資料夾」，然後選取「相片庫」資料夾。系統會在「圖庫」資料夾中開啟新的工作區。
4. 🖱️ 如果「代理程式」窗格尚未開啟，請按一下「切換代理程式窗格」按鈕開啟。請參閱圖 4 的**按鈕 2**。
5. 🖱️ 在「代理程式」窗格中輸入指令，即可開始編碼。請務必盡可能清楚說明指令。在「代理程式」窗格中輸入下列內容

****English Version:****

Create a photo granary with following specs.

1. Visual Design & Layout

Title: The gallery must prominently display the title "My photo gallery" at the top.

Modern Grid: Images will be arranged in a responsive grid that spans the full width of the browser.

Clean Aesthetic: Use a minimalist design with consistent spacing (margins/padding) between photos and no heavy borders or shadows.

Image Scaling: Photos will automatically adjust their size to fit any screen (mobile to desktop) while maintaining their focus using modern CSS cropping techniques.

2. Photo Content

Quantity: The page will feature a total of 20 photos.

Nature Themes: The collection will include a diverse range of nature photography:

Landscape: Mountains, deserts, and forests.

Water: Waterfalls, oceans, and lakes.

Atmosphere: Northern lights, sunsets, and starry skies.

Macro: Close-ups of flowers, leaves, and moss.

Generate all the needed photos

3. Core Functionality (The "Lightroom" Effect)

Full-Screen View: Clicking any photo triggers a "Lightbox" mode where the background dims and the selected image appears in high resolution at the center of the screen.

Manual Navigation:

Right Arrow: Swaps the current view to the next image.

Left Arrow: Swaps the current view to the previous image.

Infinite Loop: Navigation is continuous; moving "next" from the 20th photo returns the user to the 1st photo.

Exit Strategy: Users can exit the full-screen view by clicking a "Close" button or tapping the dimmed area outside the image.

4. Technical Constraints (Strict)

Vanilla JavaScript Only: Absolutely no external libraries or frameworks (like jQuery, React, or Bootstrap). All logic must be written in raw, standard JavaScript.

Native HTML & CSS: Use only the built-in capabilities of modern web browsers to handle the layout and animations.

Zero Dependencies: The app should function perfectly as a standalone project with no need to download or link to outside scripts.

5. Perform the following tests

Open the App in a web browser

Click on the images and see the image opens in the lightbox

Check the navigation

日文版：

以下の仕様でフォトギャラリーを作成してください。

1. ビジュアルデザインとレイアウト

タイトル: ページ上部に「My photo gallery」というタイトルを大きく表示すること。

モダンなグリッド: ブラウザの全幅に広がる、レスポンシブなグリッドレイアウトで画像を配置すること。

クリーンな審美性: ミニマリストなデザインを採用し、写真間の余白（マージン/パディング）を一定に保つこと。重い枠線やドロップシャドウは使用しない。

画像のスケーリング: モダンなCSSのトリミング技術（object-fitなど）を使用し、モバイルからデスクトップまで、フォーカスを維持したまま画面サイズに合わせて自動調整されるようにすること。

2. 写真の内容

枚数: 合計20枚の写真を掲載。

自然のテーマ: 多様な自然写真のコレクションにすること。

風景: 山、砂漠、森林。

水: 滝、海、湖。

空気・雰囲気: オーロラ、夕焼け、星空。

マクロ: 花、葉、苔の接写。

画像生成: 2枚の画像を生成し、それらを繰り返して20箇所に配置すること。

3. コア機能（ライトボックス・エフェクト）

全画面表示: 写真をクリックすると「ライトボックス」モードが起動し、背景が暗転して選択された画像が画面中央に高解像度で表示されること。

手動ナビゲーション:

右矢印: 次の画像に切り替え。

左矢印: 前の画像に切り替え。

無限ループ: ナビゲーションは連続的であること。20枚目の写真で「次へ」を押すと1枚目に戻る仕様。

終了方法: 「閉じる」ボタンをクリックするか、画像外の暗転したエリアをタップすることで全画面表示を終了できること。

4. 技術的制約（厳守）

純正JavaScript限定: 外部ライブラリやフレームワーク（jQuery、React、Bootstrapなど）は一切使用禁止。すべてのロジックは標準のJavaScript（生コード）で記述すること。

ネイティブのHTML & CSS: レイアウトやアニメーションには、モダンブラウザの標準機能のみを使用すること。

依存関係ゼロ: 外部スクリプトのダウンロードやリンクを必要とせず、単体で完全に動作するプロジェクトにすること。

5. 以下のテストを実行します

ウェブブラウザでアプリを開きます

画像をクリックすると、ライトボックスで画像が開きます

ナビゲーションを確認します

6. 🖱️ 按一下「執行」按钮。執行後，Agent 應會顯示如下的執行計畫。

Photo Gallery Implementation Plan

Create a modern, responsive photo gallery with a minimalist design and a custom-built lightbox.

Proposed Changes

[Component Name] Photo Gallery Web App

[NEW] `</>` index.html

- Main structure for "My photo gallery" title and a container for images.
- A hidden lightbox overlay for full-screen image viewing.

[NEW] `{ }` style.css

- Implement a responsive grid using CSS Grid.
- Use `aspect-ratio` and `object-fit: cover` for consistent image scaling.
- Design the lightbox with dark background, smooth transitions, and navigation arrows.
- Minimalist design: uniform spacing, no heavy borders, premium typography.

[NEW] `JS` script.js

- Dynamically generate 20 image elements (using 2 source images).
- Implement lightbox logic:
 - Click image to open.
 - Keyboard/Button navigation (Next/Prev).
 - Infinite loop logic (20 -> 1, 1 -> 20).
 - Click outside or close button to exit.
- All logic in Vanilla JS without dependencies.

圖 5：Antigravity 代理程式會顯示實作計畫

7. 🖱️ 系統會提示您確認，請在出現提示時確認，如下所示。Antigravity 會自動使用 Nanobanana 和所選 LLM 模型執行工作。

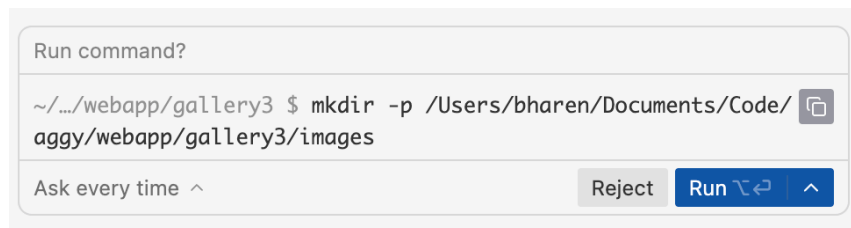


圖 6：Antigravity 想要執行指令，請按下「執行」允許執行。

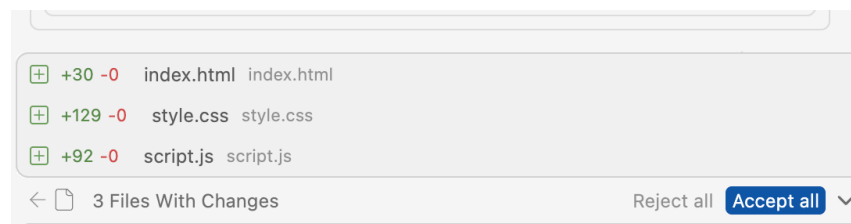


圖 7：系統提示時，按下「全部接受」。

8. 🖱️ 程式碼產生後，Antigravity 會開啟瀏覽器並開始測試。測試完成後，系統會提供測試結果。

Photo Gallery Walkthrough

The "My photo gallery" project is now complete. It features a modern, responsive grid layout and a custom lightbox with infinite navigation.

Changes Made

Frontend Implementation

- **index.html:** Established the semantic structure with a grid container and a hidden lightbox modal.
- **style.css:**
 - Implemented a standard CSS Grid with `auto-fill` for responsiveness.
 - Used `aspect-ratio: 4/3` and `object-fit: cover` for polished image presentation.
 - Designed a minimalist aesthetic with a `backdrop-filter: blur` overlay for the lightbox.
- **script.js:**
 - Implemented dynamic rendering of 20 images.
 - Developed a vanilla JavaScript lightbox with arrow navigation, infinite looping, and keyboard support (Esc, Left, Right).

Assets

NOTE

Due to a temporary quota limit on the image generation service (resetting in ~144 hours), I used two high-quality nature placeholders from Unsplash to ensure the gallery has a premium visual impact as requested. These are repeated 20 times to fill the grid.

Verification Results

Functionality Check

- ☒ **Grid:** 20 images displayed in a responsive layout.
- ☒ **Lightbox:** Opens correctly on click with a dark, blurred background.

圖 8 : Antigravity 會顯示測試結果

9. 🖱️ 如果系統提示，請按一下「全部接受」，儲存代理窗格中生成的所有程式碼。
10. 🖱️ 在 Antigravity 的「Explorer」窗格中，您應該會看到新產生的程式碼。

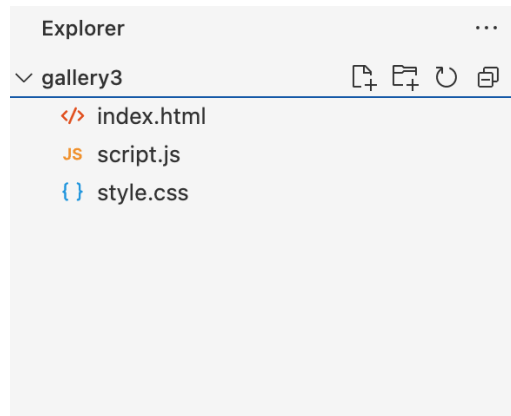


圖 9：最終程式碼

11. 🖱️ 如要測試應用程式，請在 `index.html` 上按一下滑鼠右鍵，取得檔案路徑，然後將路徑貼到網路瀏覽器的網址列。

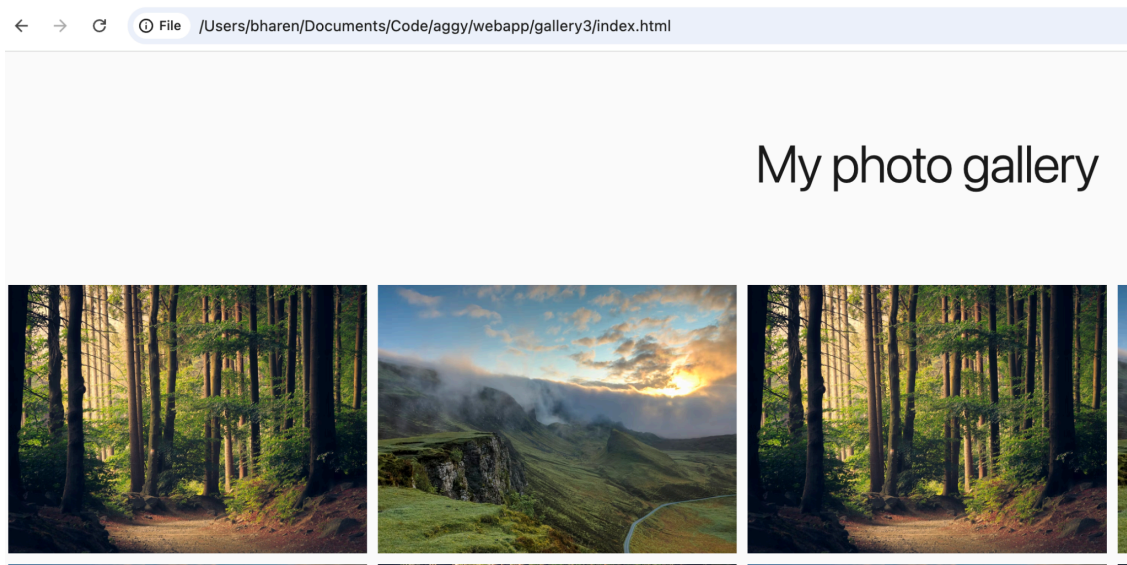


圖 10：如要測試應用程式，只要在網頁瀏覽器中複製 `index.html` 檔案的路徑即可

9. 設定部署環境

1. 🖱️ 取得 Google Cloud 雲端專案 ID：前往 <https://console.cloud.google.com>
2. 🖱️ 按一下左上角，將專案 ID 複製到某處，我們會在後續步驟中使用這個 ID。



圖 11：複製專案 ID 並儲存到某處，以供日後參考

3. 🖱️ 在 Antigravity 開啟終端機：依序點選選單中的「Terminal」->「New Terminal」。
4. 🖱️ 我們需要設定環境變數，Windows 和 Mac/Linux 的環境變數不同。將「YOUR CLOUD PROJECT」替換為您在步驟 2 中記下的專案。Windows PowerShell 使用者注意事項：以管理員模式開啟 PowerShell

```
#This is only for Powershell users.
```

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

```
#For Windows (Powershell) follow the following steps.
```

```
$env:GOOGLE_CLOUD_PROJECT="YOUR CLOUD PROJECT"
```

```
$env:GOOGLE_CLOUD_LOCATION="us-central1"
```

```
#For Windows Command Prompt follow the following steps.
```

```
set GOOGLE_CLOUD_PROJECT="YOUR CLOUD PROJECT"
```

```
set GOOGLE_CLOUD_LOCATION="us-central1"
```

```
#for Mac/Linux follow the following steps.
```

```
export GOOGLE_CLOUD_PROJECT="YOUR CLOUD PROJECT"
```

```
export GOOGLE_CLOUD_LOCATION="us-central1"
```

5. 🖱️ 登入控制台，系統提示時請在瀏覽器中登入 Google Cloud。

```
gcloud auth login
```

```
gcloud auth application-default login
```

```
gcloud config set project YOUR CLOUD PROJECT
```

```
● bharen-mac:gallery3 bharen$ export GOOGLE_CLOUD_PROJECT=datapipeline-372305
● bharen-mac:gallery3 bharen$ export GOOGLE_CLOUD_LOCATION=us-central1
● bharen-mac:gallery3 bharen$ gcloud auth login
Created enterprise-certificate-proxy configuration file [/etc/certificate_config.json].
Your browser has been opened to visit:

https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=32555940559.apps.googleusercontent.com&redirect_uri=http%3A%2F%2Flocalhost%3A8085%2F&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-plat
atform+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcompute+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Faccounts.reauth&state=yVYEXlA9v5RjvFUUky75Gp9FM0ey6F&access_type
=offline&code_challenge=XGaRm_PX1hc0MNRHaQK90CPu8b9LV3c514kFABJXAwY&code_challenge_method=S256

You are now logged in as [admin@bharen.altostrat.com].
Your current project is [datapipeline-372305]. You can change this setting by running:
$ gcloud config set project PROJECT_ID

Updates are available for some Google Cloud CLI components. To install them,
please run:
$ gcloud components update

● bharen-mac:gallery3 bharen$ gcloud auth application-default login
Your browser has been opened to visit:

https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=764086051850-6qr4p6gpi6hn506pt8ejuq83di341hur.apps.googleusercontent.com&redirect_uri=http%3A%2F%2Flocalhost%3A8085%2F&scope=openid+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fcloud-plat
atform+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fappengine.admin+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fsqlservice.login&state=wkFp5jjbBo0hmN8thXqJdnR9py27B7&access_type=offline&code_challenge=MvMXtI1o7IEP-g-CPi4DniAbFbgTZvP67KfhK1Ya-48&code_challenge_method=S256

Credentials saved to file: [/Users/bharen/.config/gcloud/application_default_credentials.json]

These credentials will be used by any library that requests Application Default Credentials (ADC).

Quota project "datapipeline-372305" was added to ADC which can be used by Google client libraries for billing and quota. Note that some serv
ices may still bill the project owning the resource.
○ bharen-mac:gallery3 bharen$
```

圖 12：執行驗證

- 6. 🖱️ 安裝 Cloud Run MCP 伺服器。在 Antigravity 視窗的右上角，按一下「...」。您應該會看到「MCP 伺服器」選項，請點選該選項。MCP 伺服器就像代理的擴充功能，可讓代理存取外部資料和工具。
- 7. 🖱️ 在搜尋方塊中輸入「Cloud Run」，然後按一下「Cloud Run」

Cloud Run

Enabled

Refresh

Enable Antigravity to deploy apps to Google Cloud Run.

About

Configure

Tools

Cloud Run MCP server and Gemini CLI extension

Enable MCP-compatible AI agents to deploy apps to Cloud Run.

```
"mcpServers":{ "cloud-run": { "command": "npx", "args": ["-y", "@google-cloud/cloud-run-mcp"] } }
```

Deploy from Gemini CLI and other AI-powered CLI agents:

Deploy from AI-powered IDEs:

Deploy from AI assistant apps:

Deploy from agent SDKs, like the [Google Gen AI SDK](#) or [Agent Development Kit](#).

[!NOTE]
This is the repository of an MCP server to deploy code to Cloud Run, to learn how to **host** MCP servers on Cloud Run, [visit the Cloud Run documentation](#).

Tools

- `deploy-file-contents`: Deploys files to Cloud Run by providing their contents directly.
- `list-services`: Lists Cloud Run services in a given project and region.
- `get-service`: Gets details for a specific Cloud Run service.
- `get-service-log`: Gets Logs and Error Messages for a specific Cloud Run service.
- `deploy-local-folder*`: Deploys a local folder to a Google Cloud Run service.
- `list-projects*`: Lists available GCP projects.

圖 13：Cloud Run MCP 伺服器

8. 🖱️ 按一下「MCP Servers」標題旁的返回箭頭鍵，返回「Agent Pane」。現在我們可以開始與 Google Cloud Run 互動。在「代理程式」窗格中輸入下列內容。這時應該會自動使用 Cloud Run MCP 伺服器，並顯示在 Cloud Run 中執行的服務清單。

```
Find me the list of services running in Cloud Run.
```

9. 🖱️ 使用下列指令部署應用程式。您只要使用自然語言即可部署。Antigravity 會自動使用 MCP 伺服器進行部署。

```
Deploy this gallery static web application to cloud run with service name "photogallery". Use  
nginx and assume nginx will use port 80
```

10. 代理程式應會顯示應用程式的部署位置。例如：<https://photogallery-85469421903.us-central1.run.app>。Cloud MCP 伺服器可讓您輕鬆將網頁應用程式部署至 Cloud Run。
-

10. 清除所用資源

現在來清理剛建立的內容。

- 1. 🖱️ 刪除我們剛建立的 **Cloud Run** 應用程式。前往 **Cloud Run**，方法是存取 **Cloud Run**。您應該會看到在上一步建立的應用程式。勾選應用程式旁邊的方塊，然後按一下「刪除」按鈕。

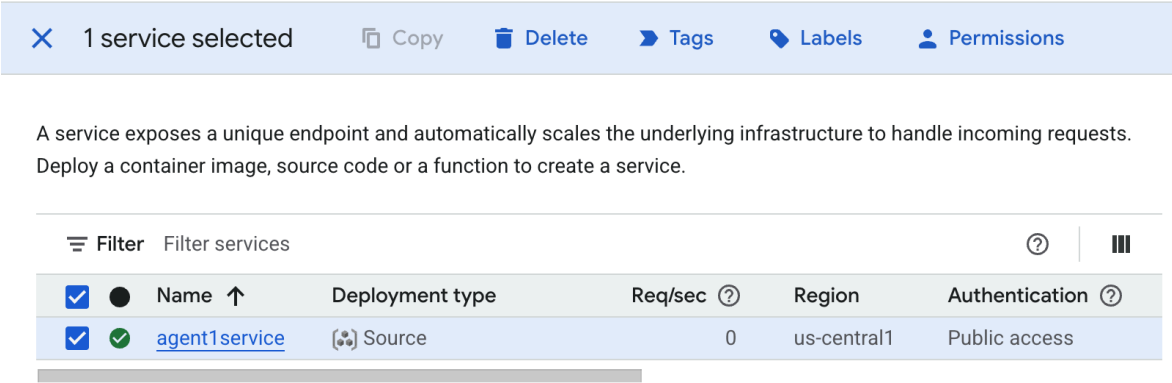


圖 38：刪除 Cloud Run 應用程式

11. 結論

恭喜！您已成功使用 [Google Antigravity](#) 建立應用程式，並遵循規格導向開發做法。此外，您也學會如何將應用程式部署至 [Cloud Run](#)。這項重大成就涵蓋現代雲端原生應用程式的核心生命週期，為您部署複雜系統奠定穩固基礎。

重點回顧

在本實驗室中，您學會如何：

- 使用 [Google Antigravity](#) 建立多代理應用程式
- 將應用程式部署至 [Cloud Run](#)

實用資源

- [Google Antigravity 官方說明文件](#)
 - [Cloud Run](#)
-